

Escola Superior de Tecnologia e Gestão de Águeda

Sistemas Operativos

Técnico Superior Profissional em Redes e Sistemas Informáticos

Gestão de Memória Virtual

Professor: Ivan Miguel Pires

2025/2026

OBJETIVOS E BIBLIOGRAFIA

Descrever os **benefícios** de um sistema de memória virtual

Explicar o conceito de **paginação a pedido**

Discutir algoritmos de **substituição** de página e esquemas de **atribuição** de frames

“Operating System Concepts”. Abraham Silberschatz, Peter B. Galvin, Greg Gagne, 9th edition, 2014, Capítulo 9

NOTA: Os acetatos que se seguem não substituem a bibliografia aqui referida, e deverão por isso ser vistos apenas como um complemento para o estudo da matéria.

MEMÓRIA VIRTUAL

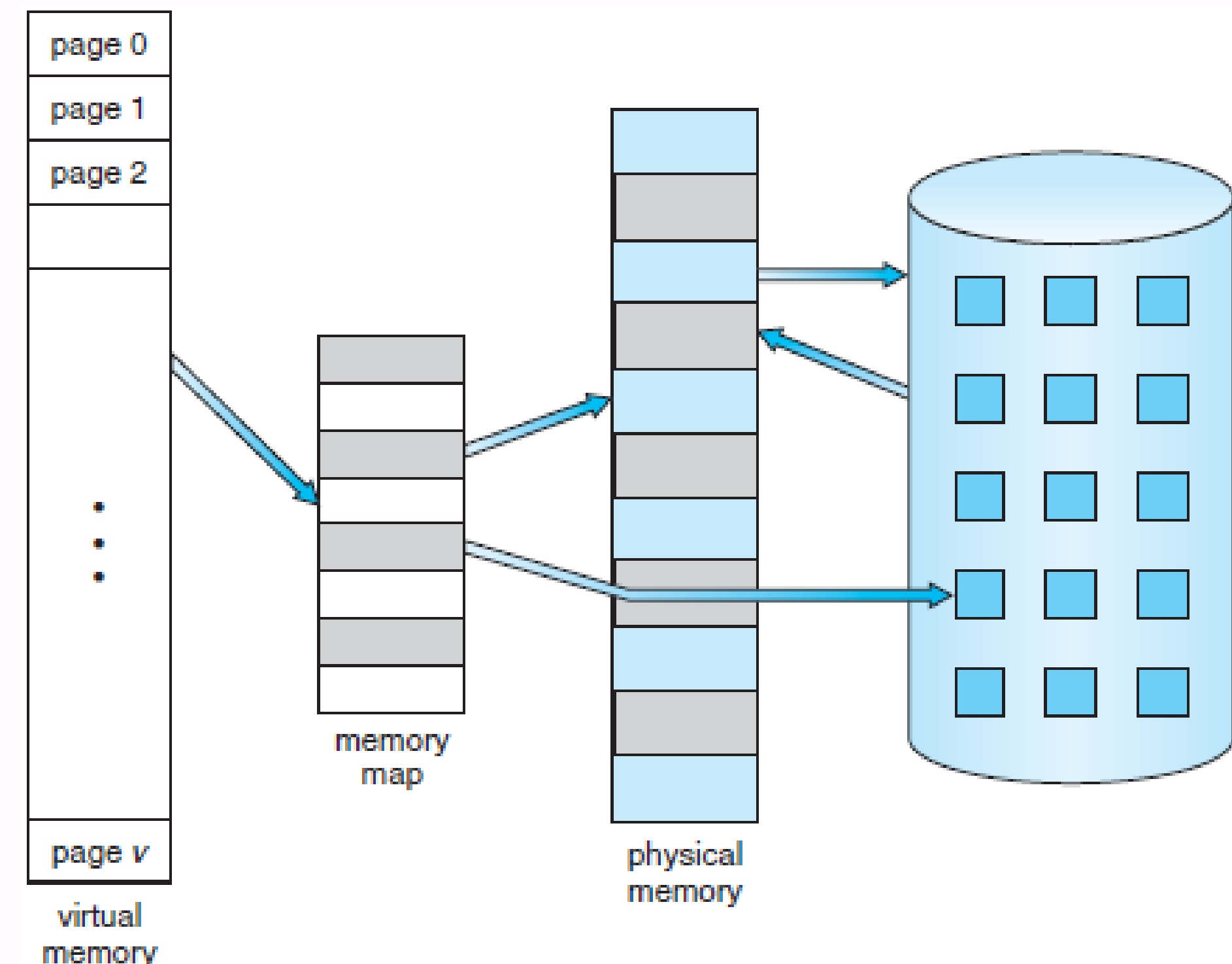
Técnica que permite que **apenas parte do programa esteja em memória** para execução

útil porque a maioria dos programas tem código que é raramente chamado

Espaço de endereçamento lógico pode assim ser **muito maior** do que o espaço de endereçamento físico

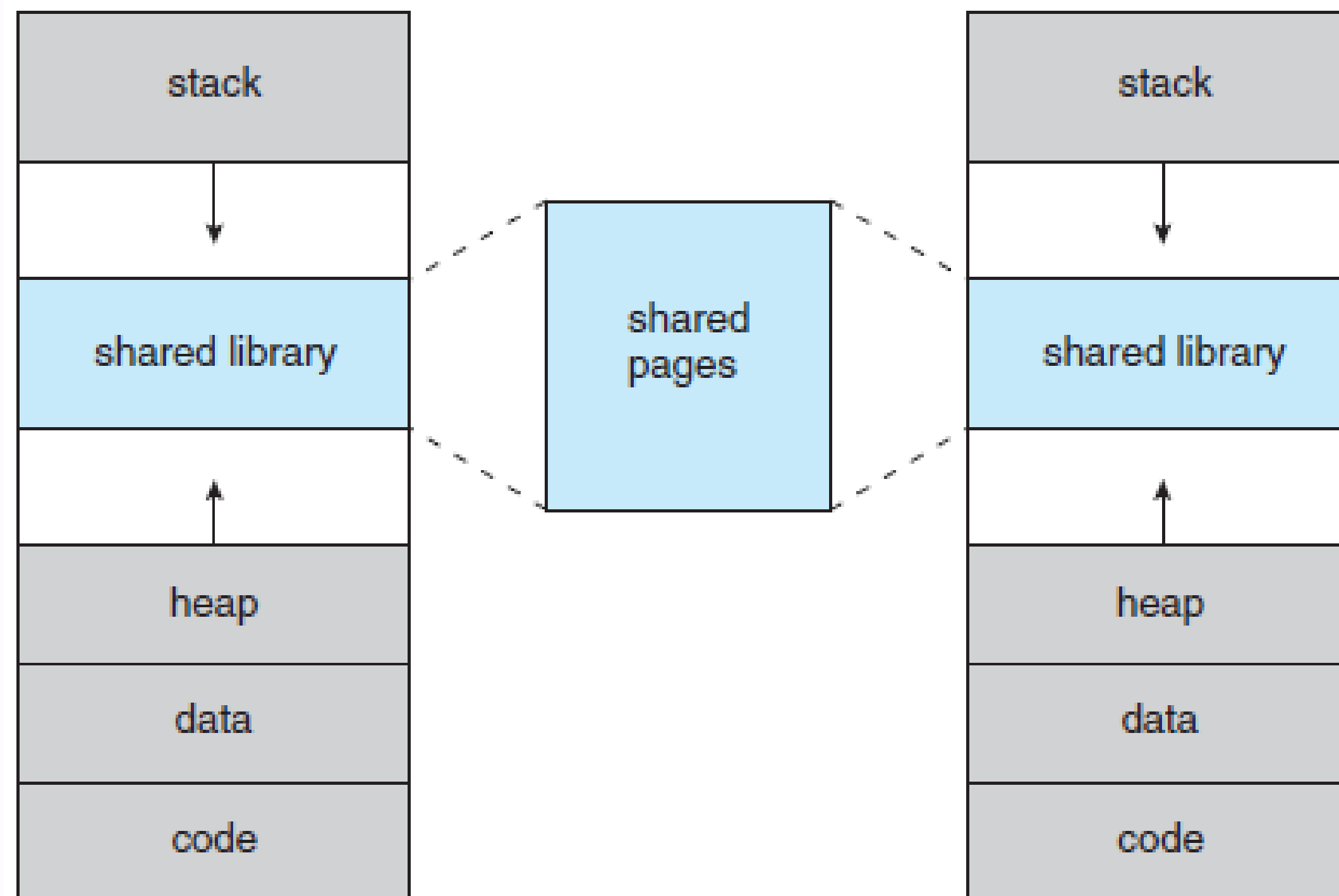
a memória física deixa de ser uma **limitação**: podemos correr programas **maiores** do que a memória disponível

como os programas ocupam menos espaço em memória, podemos ter **mais programas** a correr simultaneamente



OUTROS BENEFÍCIOS

Além de **separar** a memória lógica da memória física, a memória virtual permite a partilha de **bibliotecas** do sistema entre vários processos



a criação de espaços de **memória partilhada** entre processos

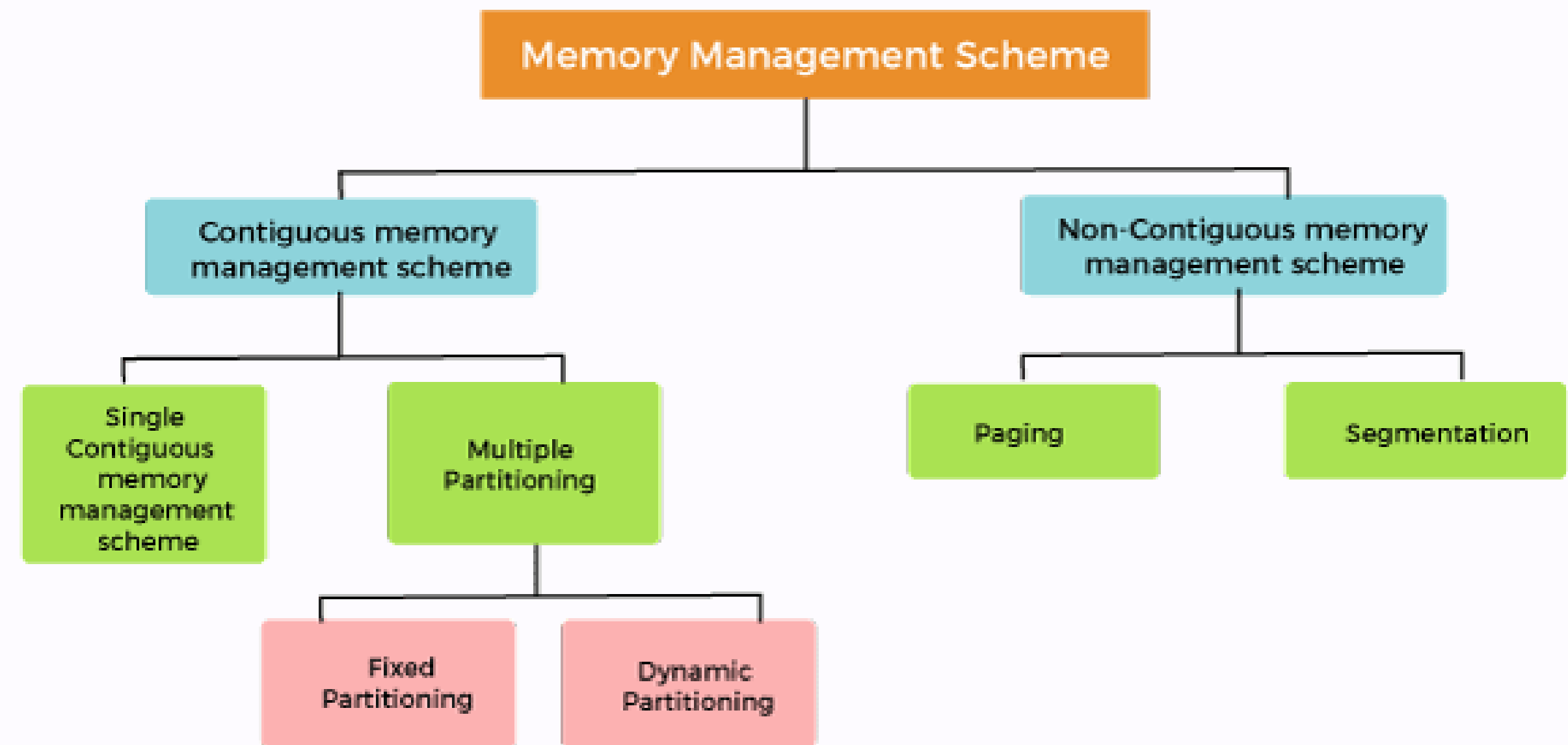
a **aceleração** da criação de novos processos (`fork()`) através da partilha de páginas

GESTÃO DE MEMÓRIA VIRTUAL: ALOCAÇÃO / RESERVA

A alocação/reserva de memória ocorre na criação (reserva de memória para código, dados e pilha), ou no pedido de extensão do espaço de endereçamento (área de dados ou pilha). A terminação de processos liberta a memória.

Alocação de páginas (blocos de dimensão fixa): qualquer página de entre as que estão livres.

- Alocação de segmentos (blocos de dimensão variável):
 - Análise da lista de blocos livres para encontrar um com dimensão igual ou superior à desejada.
 - Escolher o bloco livre que provoque menor fragmentação externa.
 - Alocar parte ou totalidade do bloco escolhido de acordo com uma dimensão mínima para o fragmento gerado. Esta dimensão mínima gera fragmentação interna mas reduz a lista de blocos livres.



Classification of memory management schemes

GESTÃO DE MEMÓRIA VIRTUAL: ALOCAÇÃO / RESERVA

Algoritmos para selecção do bloco livre (dimensão variável):

best-fit (o melhor possível): utiliza o melhor espaço de memória, ou seja, aquela que deixa o menor espaço alocado sem utilização. Uma grande desvantagem dessa estratégia é que, como são alocados primeiramente as partições menores, deixando pequenos blocos, existe maior fragmentação

Ex.: supondo os seguintes espaços de memória disponíveis para alocação: 11k, 3k, 19k, 18k, 7k, 8k, 13k, 15k.

Se o algoritmo best fit for utilizado, as solicitações 5k, 12k, 6k ocupariam os espaços 7k, 13k, 8k respectivamente

worst-fit (o maior/pior possível): aloca o programa na pior partição, ou seja, aquela que deixa o maior espaço livre.

Esta técnica, apesar de aproveitar primeiro as partições maiores, acaba por deixar espaços livres suficientemente grandes para que outros programas utilizem a memória, o que diminui ou, pelo menos, retarda a fragmentação

GESTÃO DE MEMÓRIA VIRTUAL: ALOCAÇÃO / RESERVA

Algoritmos para selecção do bloco livre (dimensão variável):

first-fit (o primeiro possível): utiliza o primeiro espaço de memória que encontrar com tamanho suficiente.

Ex.: suponhamos os seguintes espaços livres: 11k, 3k, 19k, 18k, 7k, 8k, 13k, 15k. Se o First-Fit for utilizado, as solicitações 5k, 12k, 6k, ocupariam os espaços 11k, 19k, 18k respetivamente.

next-fit (o primeiro possível a seguir ao anterior): Algoritmo para partição dinâmica que inicia a busca a partir da posição da última alocação até encontrar o primeiro bloco

Mais frequentemente são alocados blocos de tamanho grande.

Grandes blocos são particionados em blocos menores e existe a necessidade de compactação quando não houver mais memória disponível

GESTÃO DE MEMÓRIA VIRTUAL: ALOCAÇÃO / RESERVA

Algoritmos para selecção do bloco livre (dimensão variável):

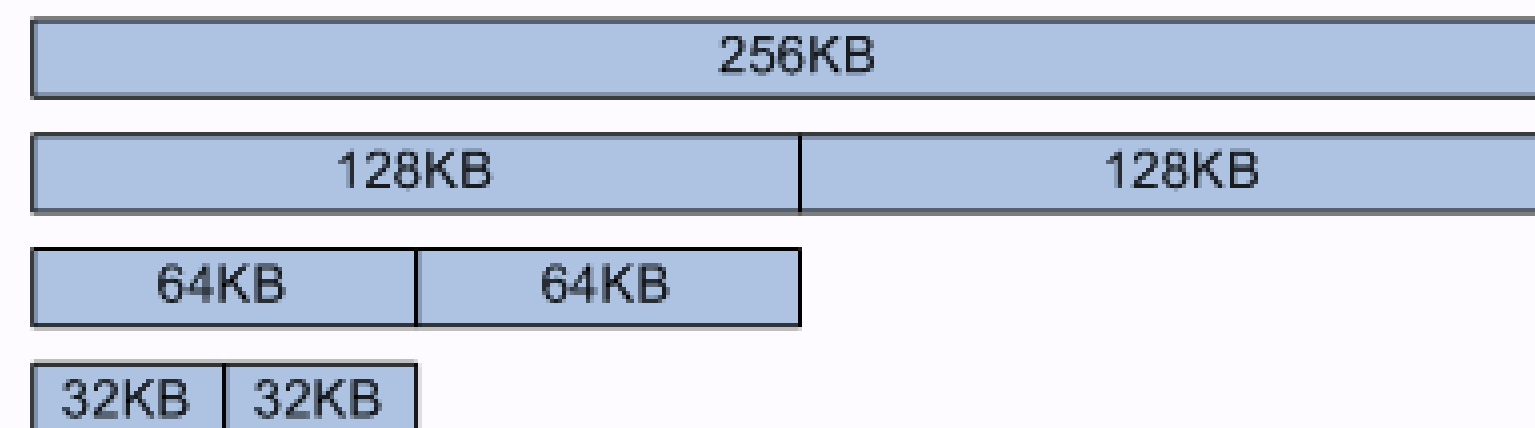
buddy: A memória livre é dividida em blocos de dimensão 2^n . Para satisfazer um pedido de dimensão R , percorre-se a lista à procura de um bloco de dimensão R tal que $2^{k-1} < R < 2^k$. Se não for encontrado procura-se um de dimensão 2^{k+i} , $i > 0$, que será dividido em duas partes iguais (buddies). Um dos buddies será subdividido quantas vezes for necessário até se obter um bloco de dimensão 2^k . Na libertação, um bloco é re combinado com o seu buddy, sendo a associação entre buddies repetida até se obter um bloco com a maior dimensão possível.

Consegue-se um bom equilíbrio entre o tempo de procura e a fragmentação interna e externa.

Fácil de implementar

Não necessita de MMU (pode ser implementado em sistemas antigos como o 80286)

Exemplo: pedido de 21 kB



GESTÃO DE MEMÓRIA VIRTUAL: TRANSFERÊNCIA

Cenários em que a transferência pode ocorrer:

on request: o programa ou o SO determinam quando se deve carregar o bloco em memória principal.

on demand: o bloco é acedido e gera-se uma falta (de segmento ou de página), sendo necessário carregá-lo para a memória principal.

prefetching: O SO recorre a heurísticas para prever quais as páginas ou segmentos que o processo vai aceder a curto prazo. Se existe espaço e a probabilidade de aceder a um bloco é elevada, transfere-se o bloco para a memória principal. Antecipa-se a ocorrência de uma falta.

GESTÃO DE MEMÓRIA VIRTUAL: TRANSFERÊNCIA

Transferência de segmentos:

Normalmente um processo precisa de ter pelo menos um segmento de código, de dados e de stack em memória.

A transferência de segmentos é normalmente feita a pedido.

Se não existir espaço disponível em memória, os segmentos de outros processos que não estejam em execução (bloqueados ou suspensos) são colocados em memória secundária (swapping). São, geralmente, usados três critérios para decidir qual o processo a transferir para disco:

- estado e prioridade do processo: processos bloqueados e pouco prioritários são candidatos preferenciais;

- tempo de permanência na memória principal: um processo tem que permanecer um determinado tempo a executar-se antes de ser novamente enviado para disco;

- dimensão do processo.

GESTÃO DE MEMÓRIA VIRTUAL: TRANSFERÊNCIA

Transferência de páginas:

A transferência de páginas é normalmente feita por necessidade. As páginas de um programa que não sejam acedidas durante a execução de um processo nunca chegam a ser carregadas em memória principal.

Na transferência por antecipação, procura-se diminuir o número de faltas de página e otimizar os acessos a disco.

Quando não existe espaço em memória principal, as páginas retiradas de memória principal são guardadas numa zona separada do disco. As páginas modificadas são transferidas em grupo para a memória secundária de modo a otimizar os acessos a disco.

GESTÃO DE MEMÓRIA VIRTUAL: SWAP AREA

É uma zona do disco para onde é transferido o conteúdo das páginas modificadas expulsas da memória:

páginas de ficheiros não alteráveis, por ex. código, não precisam ser transferidas para a swap area.

Pode ser implementada:

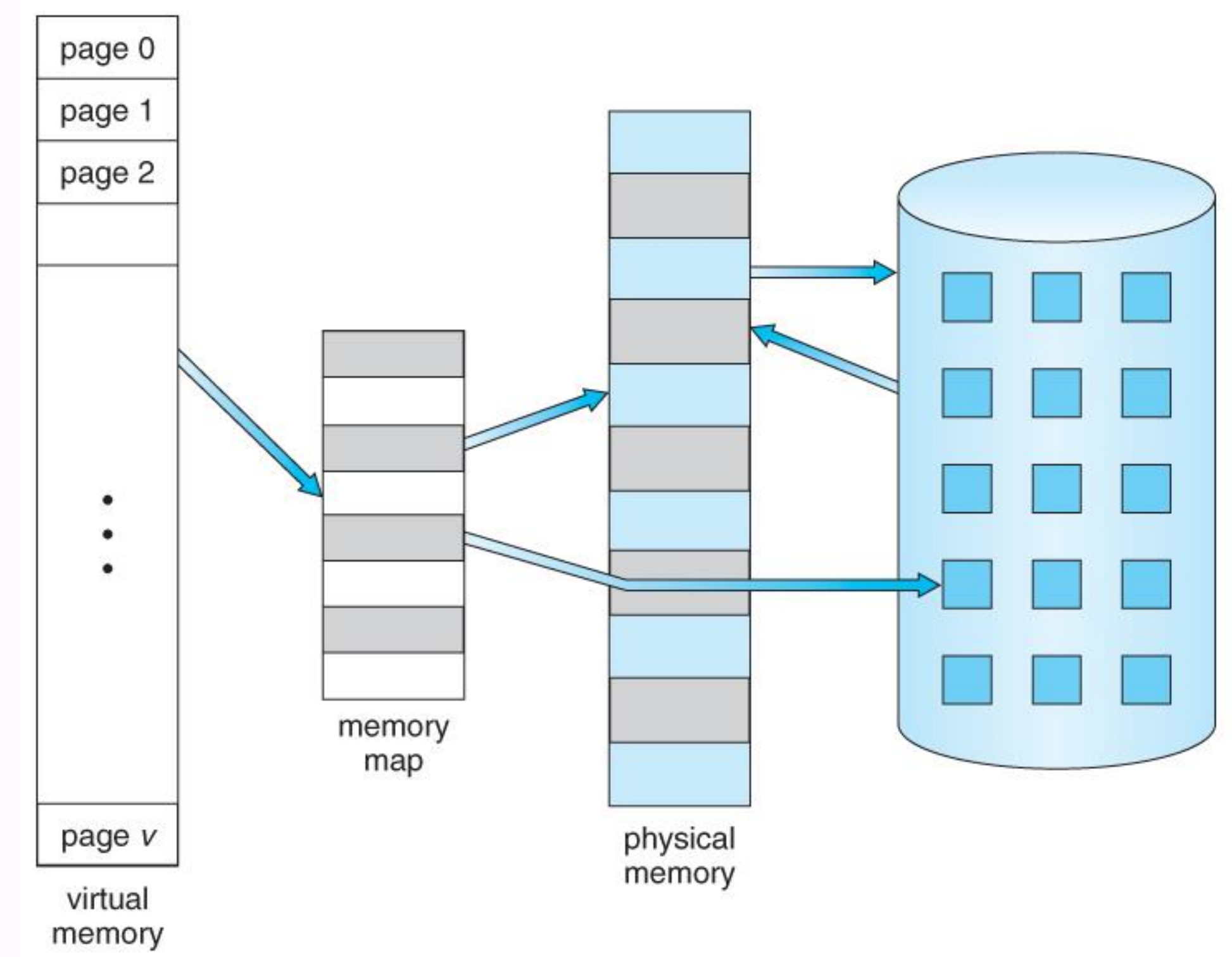
directamente sobre o disco – mais eficiente;
sobre ficheiros – fácil de ajustar o seu tamanho.

Em qualquer caso, o SO tem que gerir o espaço disponível na swap area.

A alocação de espaço na área de swap pode ser:

Otimista: só é alocada quando necessário – possibilidade de deadlock?

Pessimista: é reservada – pode não vir a ser usada e, consequentemente, reduzir a utilização dos recursos.



PAGINAÇÃO A PEDIDO (DEMAND PAGING)

A página só é colocada em memória quando

for **necessária**

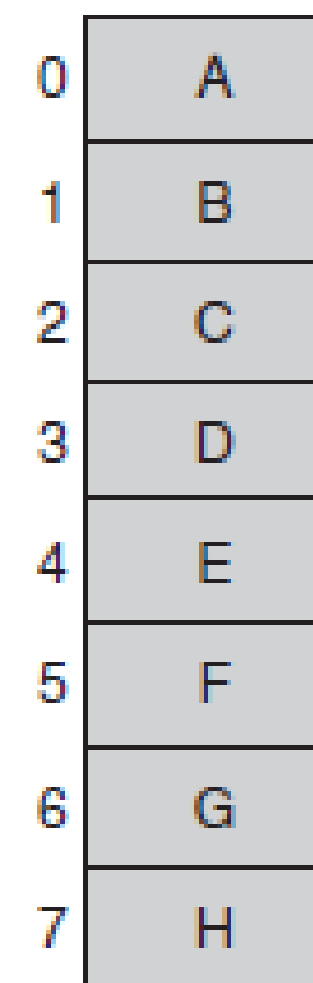
se **nunca** for necessária, **nunca** é colocada em memória

Como saber então se uma página está em memória ou no disco?

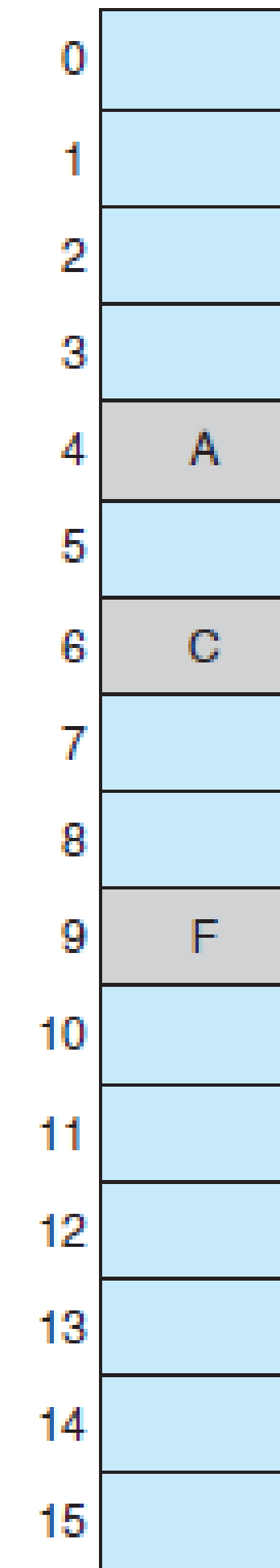
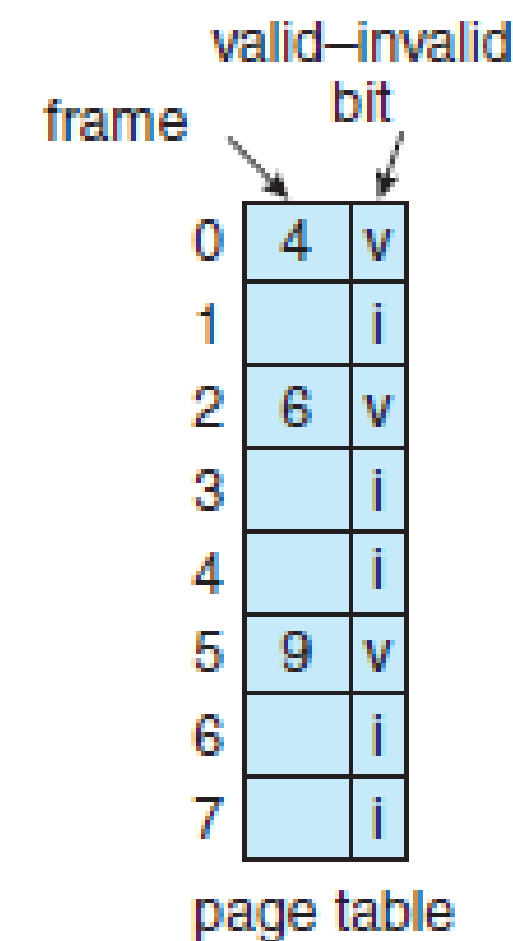
possível solução (hardware): usar o esquema do bit **válido-inválido**

se válido, página é válida e está em **memória**

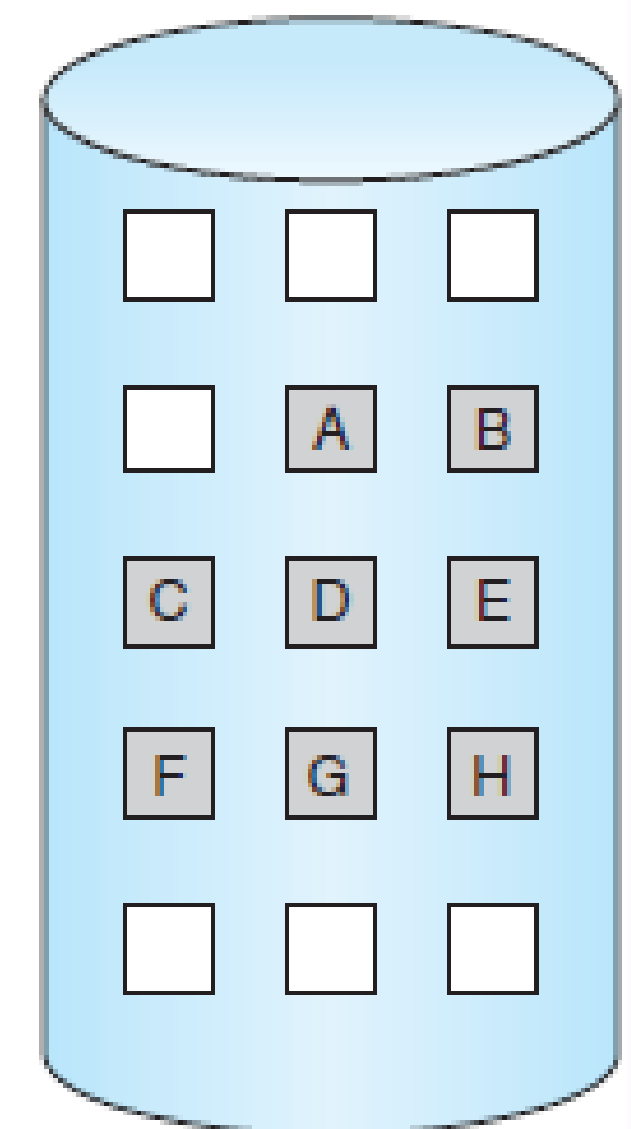
se inválido, ou não é válida, ou está no **disco**



logical memory



physical memory



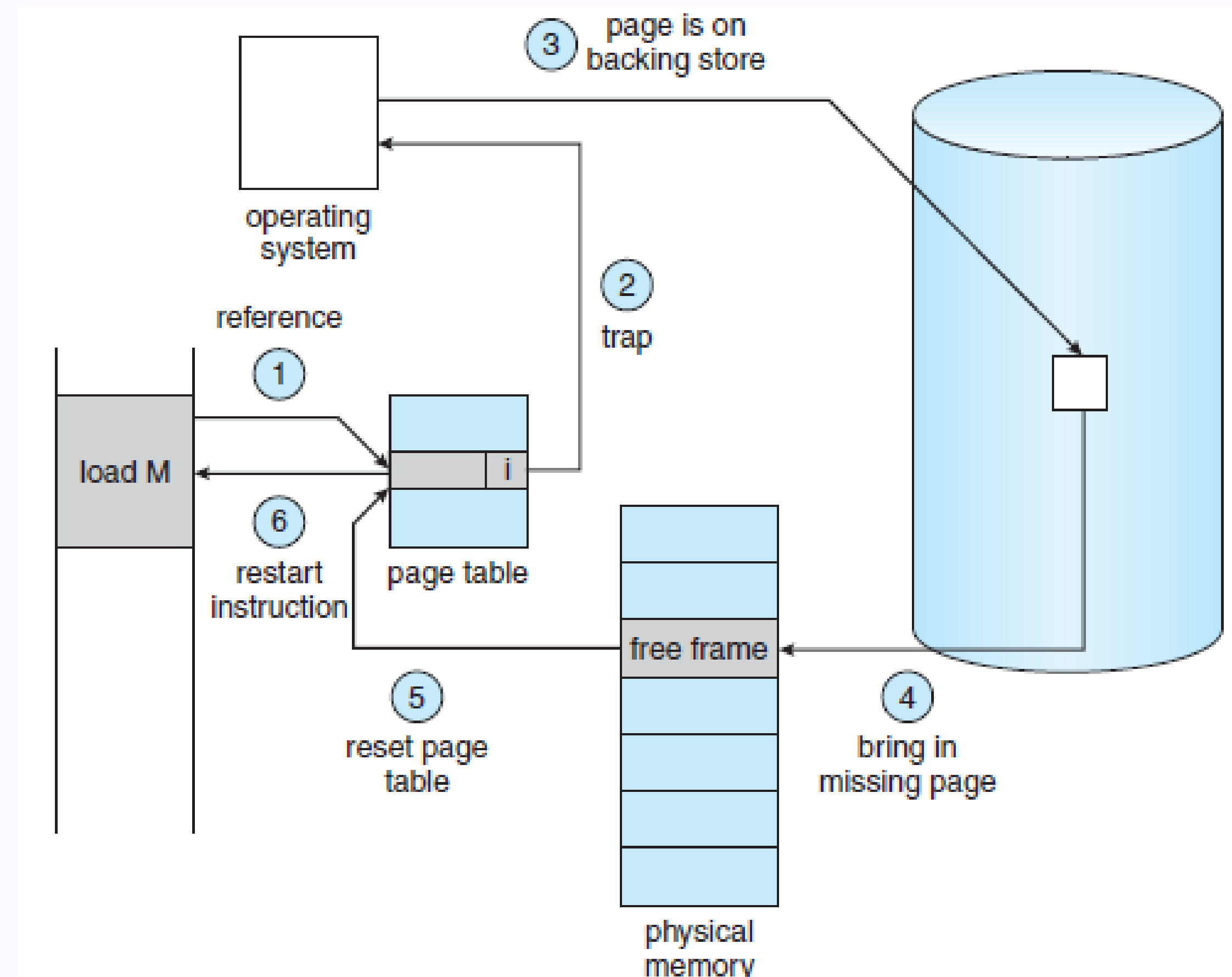
FALTA DE PÁGINA (PAGE FAULT)

Quando se tenta aceder a uma **página marcada como inválida** é causado um **page fault**

O que fazer?

- (1) SO examina **outra tabela interna** (tipicamente no PCB) e:
- (2) Se referência inválida, **termina** programa
- (3) Caso contrário, está em disco, **trazer** para memória
- (4) Obter frame vazia e colocar a página nessa **frame**
- (5) **Atualizar** as tabelas e o bit de validação
- (6) **Repetir** instrução que causou o page fault

Problema: Desempenho



PERFORMANCE OF DEMAND PAGING

Stages in Demand Paging (worse case)

1. Trap to the operating system
2. Save the user registers and process state
3. Determine that the interrupt was a page fault
4. Check that the page reference was legal and determine the location of the page on the disk
5. Issue a read from the disk to a free frame:
 1. Wait in a queue for this device until the read request is serviced
 2. Wait for the device seek and/or latency time
 3. Begin the transfer of the page to a free frame
6. While waiting, allocate the CPU to some other user
7. Receive an interrupt from the disk I/O subsystem (I/O completed)
8. Save the registers and process state for the other user
9. Determine that the interrupt was from the disk
10. Correct the page table and other tables to show page is now in memory
11. Wait for the CPU to be allocated to this process again
12. Restore the user registers, process state, and new page table, and then resume the interrupted instruction

PERFORMANCE OF DEMAND PAGING (CONT.)

Three major activities

Service the interrupt - careful coding means just several hundred instructions needed

Read the page - lots of time

Restart the process – again just a small amount of time

Page Fault Rate $0 \leq p \leq 1$

if $p = 0$ no page faults

if $p = 1$, every reference is a fault

Effective Access Time (EAT)

$$\text{EAT} = (1 - p) \times \text{memory access} + p (\text{page fault overhead} + \text{swap page out} + \text{swap page in})$$

DEMAND PAGING EXAMPLE

Memory access time = 200 nanoseconds

Average page-fault service time = 8 milliseconds

$$\text{EAT} = (1 - p) \times 200 + p (8 \text{ milliseconds})$$

$$= (1 - p) \times 200 + p \times 8,000,000$$

$$= 200 + p \times 7,999,800$$

If one access out of 1,000 causes a page fault, then

$$\text{EAT} = 8.2 \text{ microseconds.}$$

This is a slowdown by a factor of 40!!

If want performance degradation < 10 percent

$$220 > 200 + 7,999,800 \times p$$

$$20 > 7,999,800 \times p$$

$$p < .0000025$$

- < one page fault in every 400,000 memory accesses

Não sei diretamente
no teste

E SE NÃO HOUVER FRAMES LIVRES?

Escolher uma que esteja em memória sem ser usada e **substituir**

1. Método básico:
2. escrever o conteúdo da frame "**vítima**" para disco
3. atualizar bit validação
4. ler a nova página para memória atualizar tabela

Um método mais eficiente implica o uso de um **bit de modificação**

se a vítima não tiver sido modificada não é necessário escrever para disco (passo 1)

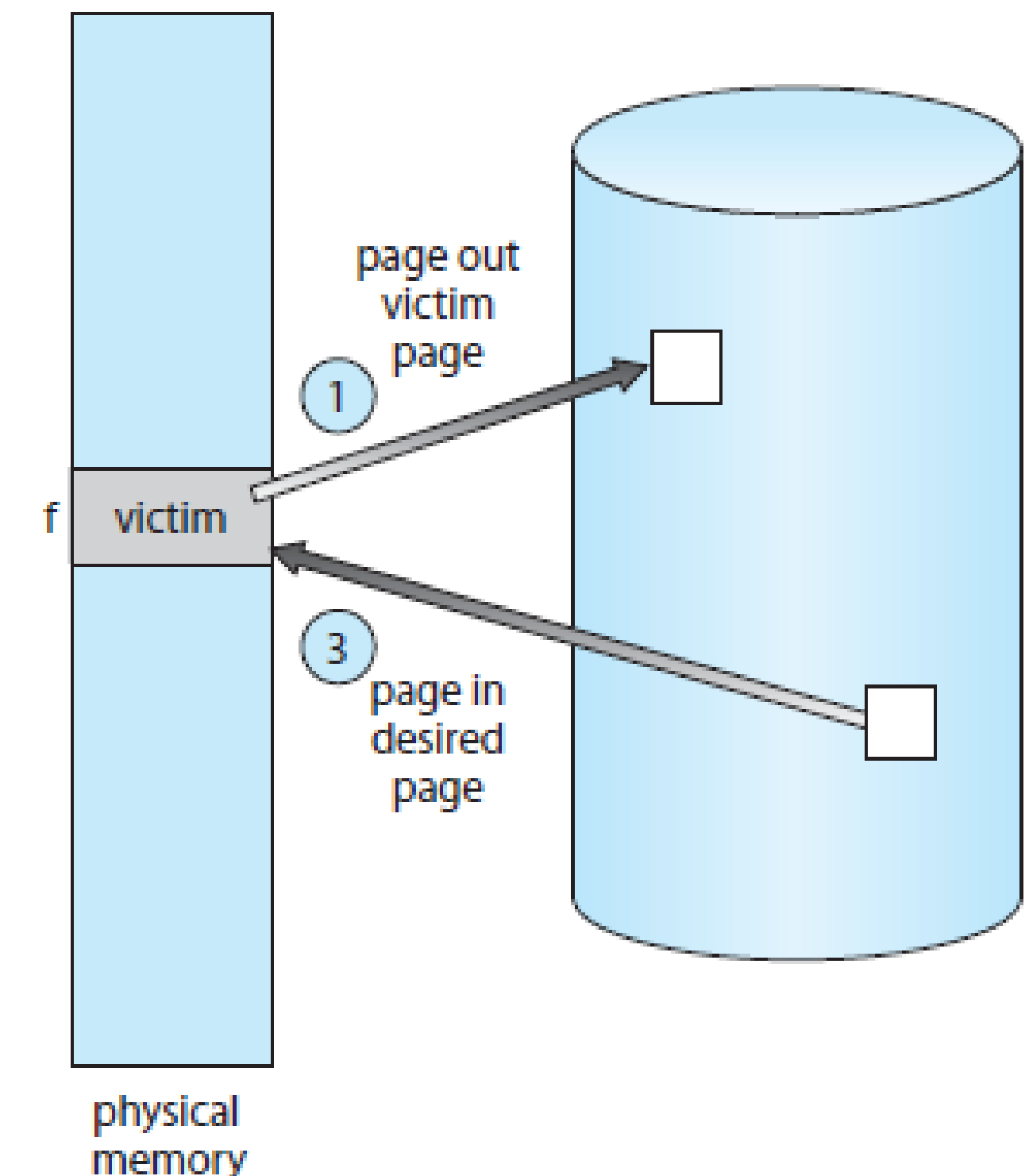
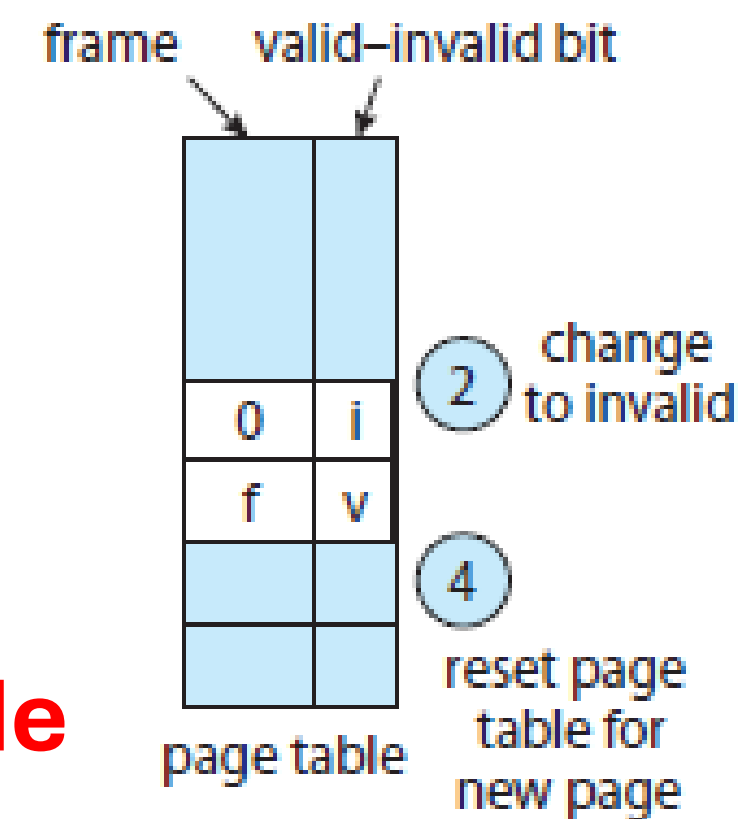
Mas que página remover?

vários **algoritmos** de substituição: FIFO, Ótimo, LRU e variantes, etc.

Qual o melhor?

o que **reduza** o número de page faults

Isso sai,
e o resto (ouvi?)



Pergunta: que algoritmo usava
numa situação e porquê?

ALGORITMO - FIFO – FIRST-IN-FIRST-OUT

O algoritmo seleciona a página que está há mais tempo no sistema

Implementação:

- Utilização de uma lista duplamente ligada. A última página em falta é colocada na primeira posição da fila. A página a ser substituída está no fim da fila (a mais antiga).
- Não atende ao princípio de localidade temporal (páginas mais recentemente usadas podem ser substituídas). A página que foi carregada há mais tempo pode estar a ser utilizada.

Exemplo:

- Anomalia de Belady (a 1.^a linha indica page fault)

- Vantagem: **simples** de implementar

- Desvantagem: performance é muitas vezes **má**, com muitos page faults

Sequência de referência:

1,	2,	3,	4,	1,	2,	5,	1,	2,	3,	4,	5
1	2	3	4	1	2	5	5	5	3	4	4
	1	2	3	4	1	2	2	2	5	3	3
		1	2	3	4	1	1	1	2	5	5

Nº de faltas de página: 9

Sequência de referência:

1,	2,	3,	4,	1,	2,	5,	1,	2,	3,	4,	5
1	2	3	4	4	4	5	1	2	3	4	5
	1	2	3	3	3	4	5	1	2	3	4
		1	2	2	2	3	4	5	1	2	3
			1	1	1	2	3	4	5	1	2

Nº de faltas de página: 10

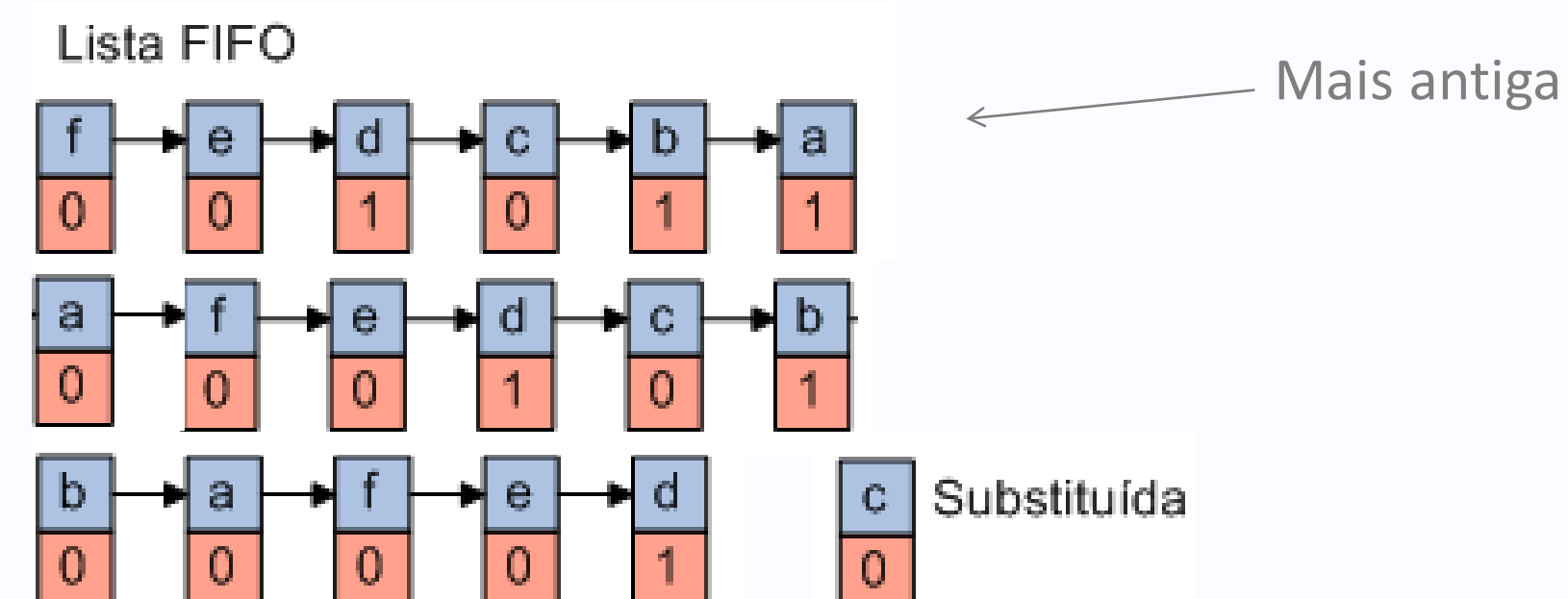
ALGORITMO - SECOND CHANCE

Algoritmo baseado no FIFO, mas utiliza o bit de referência (R) colocado a 1 se a página foi referenciada

- Antes de uma página ser substituída, se o bit R está a 1, é colocado a 0, mas a página não é substituída. A página a substituir será a primeira que for encontrada de entre as mais antigas com R=0.

■ Exemplo:

- Se R=0, a página é retirada da memória
- Caso contrário R é colocado a 0 e dá-se uma nova hipótese à página colocando-a no final da lista



ALGORITMO LEAST-RECENTLY-USED (LRU)

O algoritmo seleciona a página menos usada recentemente

- Princípio de localidade temporal.

- **Implementação:**

- Utilização de uma marca temporal do último acesso na tabela de páginas. Aquando da necessidade de substituição, deve ser feita a pesquisa da página com o menor valor.
- Manter uma pilha (lista duplamente ligada das páginas referenciadas), colocando no topo a última página referenciada. Não envolve pesquisa aquando da substituição, mas aumenta o esforço computacional associado a cada acesso.

- **Exemplo:**

Sequência de referência:

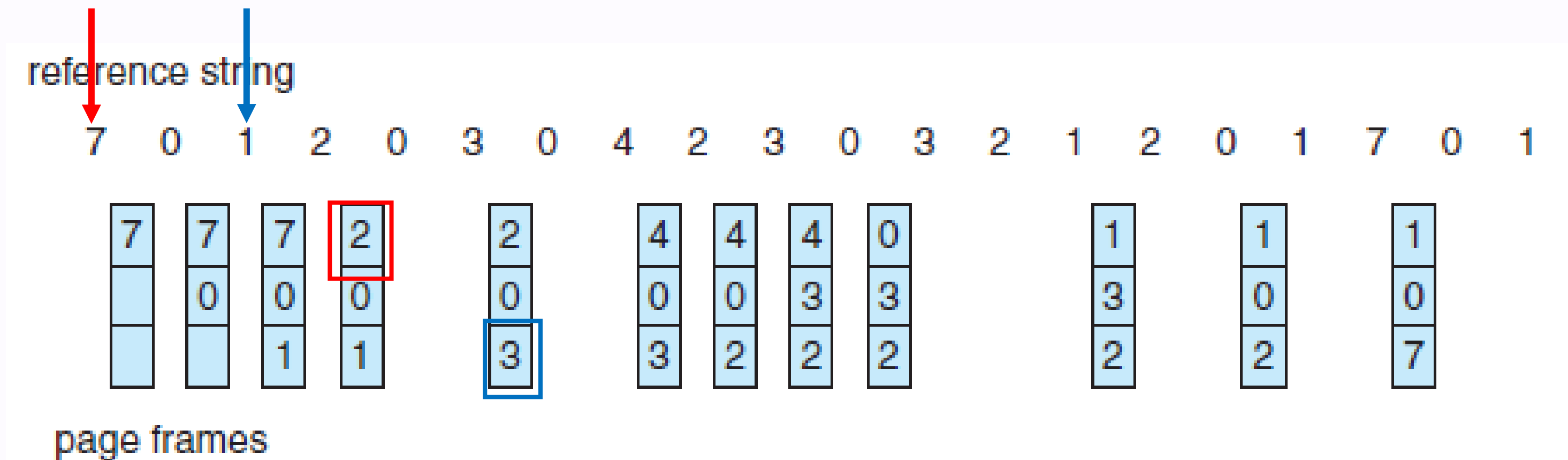
1,	2,	3,	4,	1,	2,	5,	1,	2,	3,	4,	5
1	1	1	1	1	1	1	1	1	1	1	5
	2	2	2	2	2	2	2	2	2	2	2
		3	3	3	3	5	5	5	5	4	4
			4	4	4	4	4	4	3	3	3

Nº de faltas de página: 8

ALGORITMO LEAST-RECENTLY-USED (LRU)

Substituir a página que não é usada **há mais tempo**

o **passado** é usado como antecipação do **futuro**...



Vantagem: normalmente a **performance** é boa

Desvantagem: eficiência requer **bastante suporte hardware**

- pode usar-se um **contador** a servir de relógio, incrementando-o sempre que haja acessos à memória e copiando o seu valor para a página acedida
- a página que tiver um valor mais pequeno é removida (é a mais "antiga")
- mas isso atrasa bastante os acessos à memória

ALGORITMO LEAST-FREQUENTLY-USED (LFU)

O algoritmo seleciona a página menos frequentemente referenciada

- Baseado na heurística de que uma página não frequentemente referenciada não será provavelmente referenciada no futuro.
- Aproximação do algoritmo LRU, mas com menos esforço computacional.
- **Implementação:**
 - É associado um contador a cada página carregada em memória, inicializado a zero quando a página é carregada. Sempre que ocorre uma interrupção do relógio, e para cada página, soma-se o valor do bit R (página referenciada) ao contador. É substituída a página de menor valor do contador.
- Pode, no entanto, uma página muito referenciada no passado, nunca mais ser referenciada no futuro e manter-se-á na memória principal por ter um valor elevado no contador.

ALGORITMO LEAST-FREQUENTLY-USED (LFU)

- O algoritmo seleciona a página não usada recentemente, é uma aproximação do algoritmo LRU, mas com menos esforço computacional.

- **Implementação:**

- Para cada página são utilizados dois bits: bit de página referenciada (R) e bit de página modificada (M) que permitem classificar as páginas em quatro grupos:

- 0: (R = 0, M = 0) não referenciada, não modificada (melhor escolha)

- 1: (R = 0, M = 1) não referenciada, modificada

- 2: (R = 1, M = 0) referenciada, não modificada

- 3: (R = 1, M = 1) referenciada, modificada (pior escolha)

- São substituídas as páginas dos grupos com o número mais baixo.

- Periodicamente, um processo coloca a zero o bit R, bem como, grava em disco as páginas modificadas, colocando o bit M a 0.

QUANTAS FRAMES A ATRIBUIR A CADA PROCESSO?

- No esquema de demand paging “puro” as páginas são colocadas em memória só quando são necessárias, **uma a uma**

- quando um processo se inicia será despoletada uma série de page faults, reduzindo o **desempenho** do sistema

- Na prática, um processo pode ter um certo número de páginas

- o número **máximo de páginas** está limitado pela memória física

- mas deveria de ter um número **mínimo** de páginas

- por razões de **desempenho** e porque certas instruções podem estar em **várias** páginas

- Esquemas de atribuição de frames

- atribuição **fixa**

- Uniforme - número de frames **igual** para todos os processos, independentemente do seu tamanho

- Proporcional - número de frames **proporcional** à dimensão do processo

- atribuição **por prioridade do processo**

- Maior prioridade = mais frames

SUBSTITUIÇÃO LOCAL VS GLOBAL

▪Substituição global

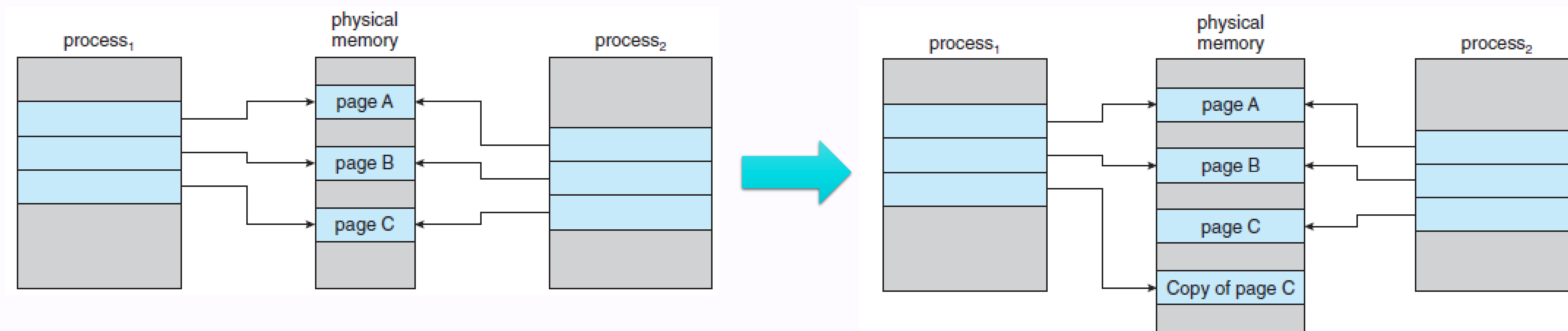
- A frame de substituição é escolhida do conjunto de **todas** as frames, podendo por isso pertencer a **outro** processo
- Melhor desempenho (normalmente), mas processo deixa de ter controlo sobre a sua taxa de page faults

▪Substituição local

- A frame de substituição é escolhida do conjunto das frames do **próprio** processo
- Assim, o número de frames atribuídas a cada processo mantém-se **inalterado**

COPY-ON-WRITE (COW)

- A chamada `fork()` cria um processo filho que é uma **cópia** do pai
 - **causa a duplicação** das páginas usadas pelo pai
 - seria de evitar essa duplicação, até porque muitas vezes os filhos chamam logo a seguir um `exec()` para correr um novo programa
- O esquema COW permite que o processo pai e o processo filho **partilhem** as mesmas páginas de memória inicialmente
 - só quando um dos processos **altera** uma página é que a página é copiada
- Criação de processos mais **rápida** e mais **eficiente**



FICHEIROS MAPEADOS EM MEMÓRIA

- Ao mapear um ficheiro do disco para memória o acesso a esse ficheiro pode ser tratado como um **normal** acesso à memória
- O ficheiro é inicialmente lido usando demand paging (resultando num page-fault)
 - parte do ficheiro (normalmente do tamanho de uma página) é depois lida do sistema de ficheiros **para memória**
 - as leituras/escritas subsequentes são tratadas como acesso normal à memória
- Vantagens
 - **simplifica** acesso ao ficheiro (é mais **fácil** e mais **rápido** aceder através da memória do que com as chamadas ao sistema `read()` e `write()`)
 - permite que vários processos mapeiem o mesmo ficheiro e assim partilhem um pedaço de memória (comunicação através de **memória partilhada**)

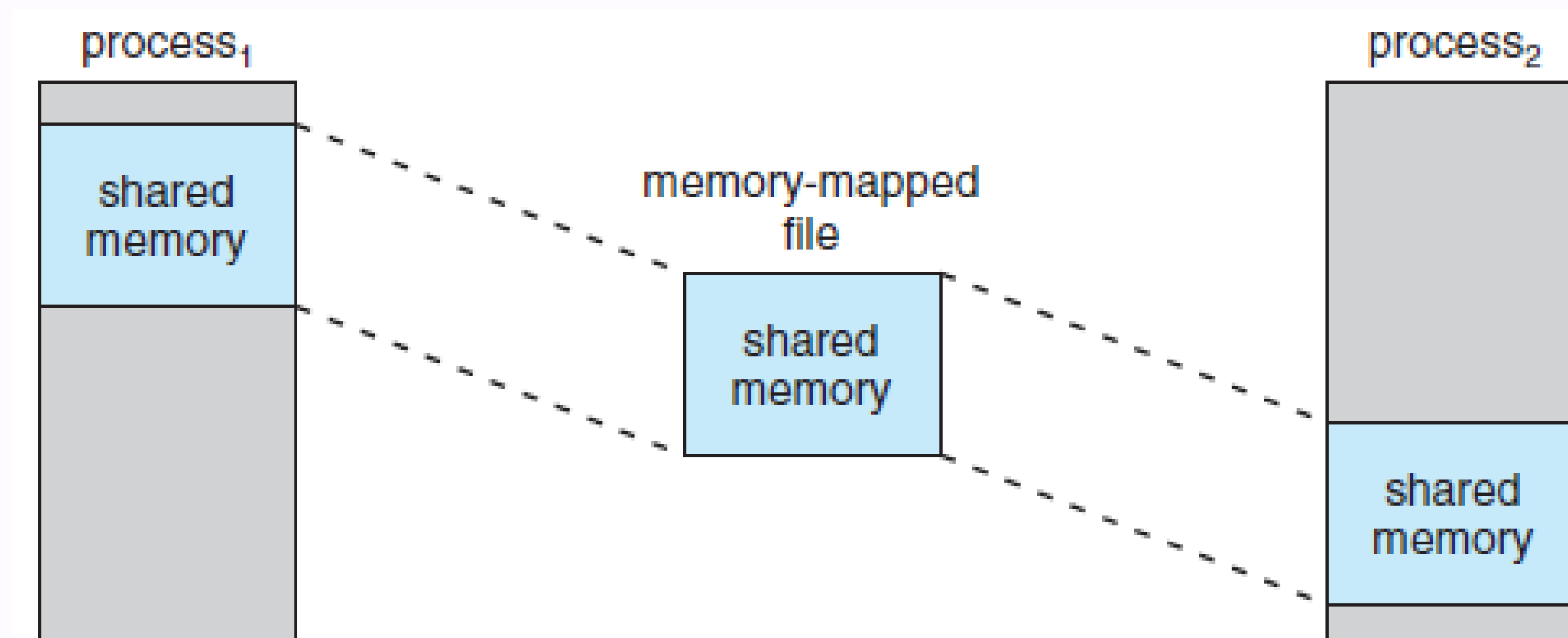


Figure 9.23 Shared memory using memory-mapped I/O.

THRASHING

- Se um processo não tem as páginas **suficientes** para executar, o número de page-faults torna-se muito elevado

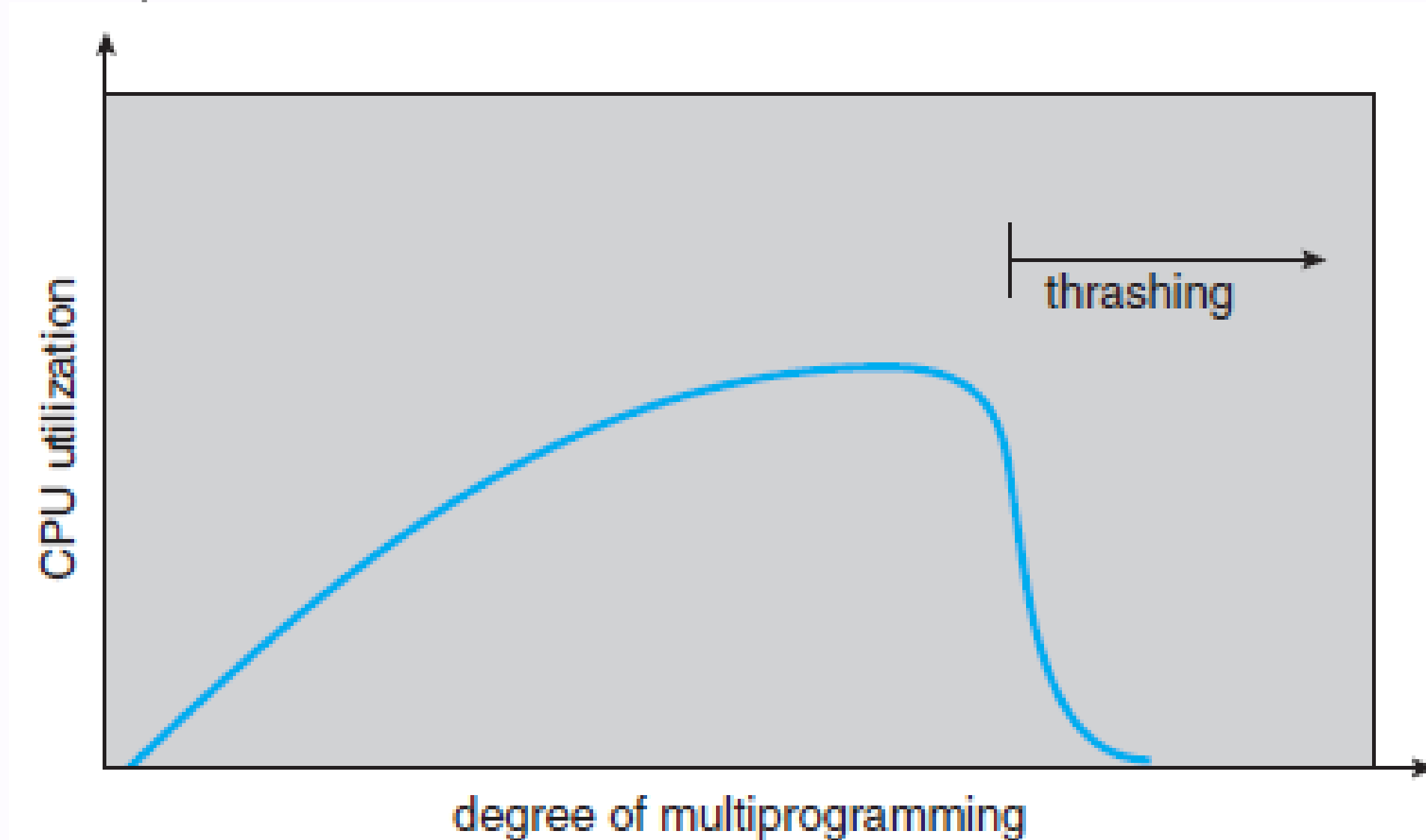
- com a consequência de haver uma **baixa utilização** do CPU
- que por sua vez leva o SO a pensar que necessita de **aumentar** o nível de multiprogramação, **adicionando** mais um processo ao sistema → efeito "bola de neve"

- **Thrashing**

- os processos passam mais tempo a resolver faltas de página do que a executar

- **Como resolver o problema?**

- um algoritmo de **substituição local limita** o problema
- melhor: fornecer aos processos as frames de que ele **necessita**!
- Questão a resolver: **quantas** frames são necessárias?



LOCALIDADE

Porque é que o esquema de demand paging funciona?

por causa da **localidade**, i.e., o processo só usa um subconjunto de páginas ativas

ao executar, os processos **migram** de uma localidade para outra

NB: as localidades podem sobrepor-se

Porque é que ocorre thrashing?

tamanho das localidades

>

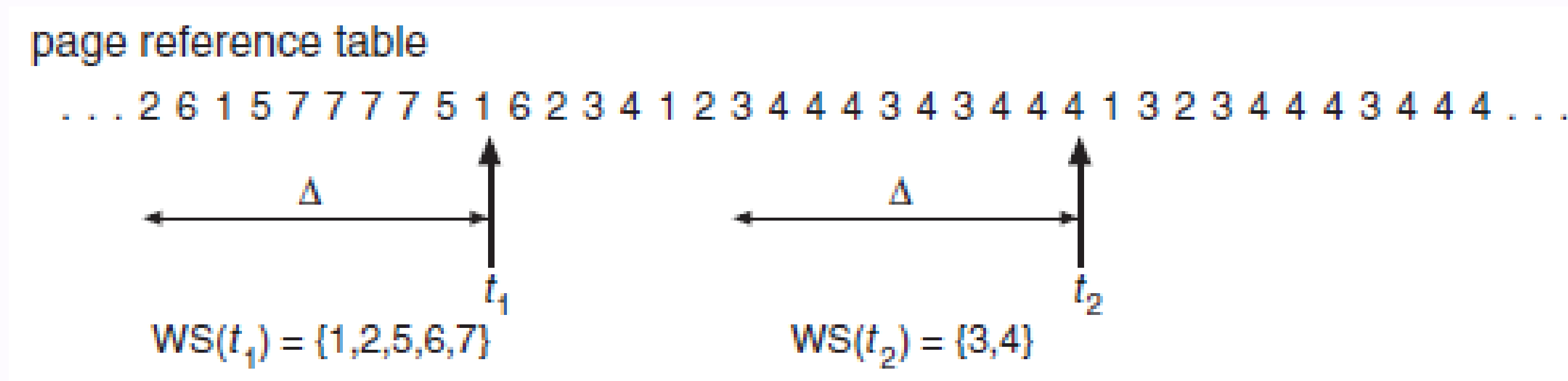
tamanho da memória

GESTÃO DE MEMÓRIA VIRTUAL: WORKING SETS

- A substituição de páginas deve ser feita globalmente, para todos os processos, ou apenas sobre as páginas do processo que gerou a falta de página?
- Com paginação pura, as páginas dum processo só são trazidas para a memória quando o processo tem uma page-fault.
- Quando um processo inicia, tem uma taxa de page-faults elevada, até que eventualmente essa taxa diminui:
 - **este comportamento deve-se à localidade de referência.**
- O conjunto de páginas acedidas por um processo num determinado intervalo designa-se por working set.
- Se o working set dum processo estiver em memória, um processo tem uma taxa de page-faults muito baixa, reciprocamente . . .
- Tipicamente, o working set dum processo varia no tempo.

MODELO WORKING-SET

- Uma solução para a questão "**quantas** frames são necessárias?"
- Este modelo baseia-se nas localidades
 - o parâmetro Δ (**janela do working-set**) mantém um número fixo de referências a páginas, sendo uma **aproximação** da localidade
 - o working set de um processo é o conjunto de páginas que foram referenciadas na **janela mais recente**



- Se o **somatório** de todos os working-sets for superior ao tamanho da memória disponível → thrashing
 - Que fazer? **Suspender** um dos processos
- Problema 1: qual o **tamanho da janela**?
 - se Δ pequeno não abrange toda a localidade
 - se Δ grande pode abranger mais do que uma localidade
- Problema 2: como **manter o working-set**?
 - cada referência à memória altera o working-set...

TAMANHO DAS PÁGINAS

- Permite **reduzir** o número elevado de page faults que ocorrem quando o processo começa ou reinicia a execução
- A ideia é **pré-paginar** algumas das páginas de que o processo pode vir a precisar
 - por exemplo, mantendo informação sobre o working-set do processo quando este é suspenso
- Dependo da utilização ou não das páginas que se pré-paginar, esta técnica trará ou não benefícios ao desempenho

PREPAGING

- É difícil à partida saber qual é o melhor tamanho de página num sistema que suporta muitas aplicações diferente
- Aspectos a ter em consideração:
 - Fragmentação: melhor páginas **pequenas** porque leva a menor desperdício de memória
 - Tamanho da tabela de páginas: melhor páginas **grandes** porque faz com que a tabela seja mais pequena
 - Localidade: melhor páginas **pequenas** porque existe melhor resolução, e logo evita-se transferir partes que não estão a ser usadas
 - Overhead dos sistemas de E/S: melhor páginas **grandes** porque os tempos de latência e de procura são muito maiores do que o tempo de transferência da informação

ALCANCE DO TLB

- O TLB é uma "cache" (em hardware) que permite traduções de endereço **rápidas**
 - evita a ida à tabela de páginas
- Para melhorar o **desempenho** do sistema, deve tentar resolver-se o maior número de endereços possíveis no TLB
 - isto é, ter um hit ratio elevado
- **Alcance do TLB**: quantidade de memória acessível via TLB
 - Alcance TLB = (Tamanho do TLB) X (Tamanho da página)
 - idealmente, o working-set de cada processo estaria no TLB.
- Como aumentar o alcance do TLB?
 - aumentando o **número de entradas** do TLB
 - Problema: TLB é hardware caro
 - aumentando o **tamanho** das páginas
 - Problema: fragmentação interna

ESTRUTURA DOS PROGRAMAS

- Normalmente a paginação a pedido deve ser um processo **transparente** para o programa do utilizador

- no entanto, há casos em que o **desempenho** do sistema pode ser melhorado substancialmente se o utilizador (ou compilador) tiver **conhecimento** acerca do sistema de paginação a pedido usado pelo SO

- Exemplo:

- assumir que cada **linha** da matriz ocupa uma página

```
int i, j;
```

```
int[128][128] data;
```

Program 1

```
for (j = 0; j < 128; j++)  
    for (i = 0; i < 128; i++)  
        data[i][j] = 0;
```

row major → that is, the array is stored

data[0][0], data[0][1], ..., data[0][127], data[1][0], data[1][1], · ·

·, data[127][127]

128 × 128 = 16,384 page faults

Programa 2

```
for (i = 0; i < 128; i++)  
    for (j = 0; j < 128; j++)  
        data[i][j] = 0;
```

This code zeros all the words on one page before starting the

next page, reducing the number of **page faults to 128**.

QUESTÕES?